

Chapter 4: Web Services and APIs

Course: Web Application Programming (ENCT 302) **Program:** BE Computer Engineering | Tribhuvan University

College: National College of Engineering

Table of Contents

- 1. [Introduction to APIs](#)
- 2. [REST Principles](#)
- 3. [RESTful API Design](#)
- 4. [JSON and XML](#)
- 5. [Data Validation and Serialization](#)
- 6. [Microservices Architecture](#)
- 7. [Building and Testing REST APIs](#)
- 8. [Model Exam Questions & Answers](#)

1. Introduction to APIs

What is an API?

An **API (Application Programming Interface)** is a set of rules and protocols that allows different software applications to communicate with each other. It defines the methods and data formats that applications can use to request and exchange information.

Simple Analogy: An API is like a waiter in a restaurant. You (client) place an order (request), the waiter (API) takes it to the kitchen (server), and brings back the food (response). You never need to know how the kitchen works.

Types of APIs

Type	Description	Example
REST API	Uses HTTP, stateless, resource-based	Twitter API, GitHub API
SOAP API	XML-based, strict contract, WS-* standards	Banking, enterprise services
GraphQL API	Query language, client defines shape	Facebook Graph API
WebSocket API	Bidirectional, persistent connection	Chat apps, live feeds
gRPC	Binary protocol, high performance	Microservices internal comms

Roles of APIs in Web Development

- 1. **Frontend-Backend Communication:** React/Angular frontend calls Django/Node backend APIs
- 2. **Third-Party Integration:** Payment gateways (eSewa, Khalti), maps (Google Maps), SMS
- 3. **Microservices Communication:** Services in a microservices architecture talk via APIs
- 4. **Mobile App Backend:** Mobile apps consume the same REST APIs as the web frontend
- 5. **Public Data Access:** Weather APIs, currency exchange APIs, government data portals
- 6. **Business Logic Exposure:** Exposing core business capabilities to partners

Web Services vs APIs

Feature	Web Service	API
Transport	Always uses a network (HTTP/SOAP)	Can be local or network
Format	XML (SOAP), JSON/XML (REST)	Any format
Scope	Subset of APIs	Broader concept
Complexity	SOAP is complex, REST is simple	Varies

2. REST Principles

What is REST?

REST (Representational State Transfer) is an architectural style for designing distributed hypermedia systems, introduced by Roy Fielding in his 2000 doctoral dissertation.

A system that follows REST principles is called **RESTful**.

The 6 REST Constraints

Constraint 1: Client-Server Architecture

- **Definition:** The client and server are separate, independent components that communicate over a network.
- **Client responsibilities:** User interface, user experience
- **Server responsibilities:** Data storage, business logic, security
- **Benefit:** Independent evolution — you can upgrade the mobile app (client) without changing the backend (server), and vice versa.

```
[React Frontend] <--HTTP--> [Django REST API] <---> [PostgreSQL Database]
      Client                Server                Storage
```

Constraint 2: Statelessness

- **Definition:** Each HTTP request from client to server must contain ALL information needed to understand and process that request. The server stores NO session state between requests.
- **Implication:** Authentication tokens (JWT) must be sent with every request; server cannot rely on "remembering" the previous request.
- **Benefit:** Scalability — any server instance can handle any request (enables load balancing).

```
# Every request is self-contained:
GET /api/orders/123 HTTP/1.1
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Constraint 3: Cacheability

- **Definition:** Responses must be labeled as cacheable or non-cacheable. If cacheable, the client (or intermediate proxy) can reuse that response for equivalent future requests.
- **HTTP Cache Headers:**
 - `Cache-Control: max-age=3600` — cache for 1 hour
 - `Cache-Control: no-cache` — always revalidate
 - `Cache-Control: no-store` — never cache (sensitive data)

- ETag: "abc123" — version identifier for conditional requests
- **Benefit:** Reduces server load and latency; improves performance.

Constraint 4: Uniform Interface

The most distinctive constraint of REST. It has 4 sub-constraints:

1. **Resource Identification in Requests:** Resources are identified by URIs. The representation sent to the client may differ from how it's stored internally.

```
URI: /api/users/42
Stored as: SQL row in users table
Sent as: JSON object
```

2. **Resource Manipulation Through Representations:** Client holds a representation (JSON/XML) and uses it to manipulate the resource (create, update, delete).
3. **Self-Descriptive Messages:** Each message includes enough information to describe how to process it (Content-Type, status codes, etc.).
4. **HATEOAS (Hypermedia as the Engine of Application State):** Responses include links to related actions/resources, allowing clients to navigate the API dynamically.

```
{
  "id": 42,
  "name": "John Doe",
  "links": {
    "self": "/api/users/42",
    "orders": "/api/users/42/orders",
    "delete": "/api/users/42"
  }
}
```

Constraint 5: Layered System

- **Definition:** The client cannot tell whether it is connected directly to the end server or an intermediary (load balancer, cache, security gateway).
- **Layers may include:**
 - API Gateway (authentication, rate limiting)
 - Load Balancer (distribute traffic)
 - CDN Cache (static content)
 - Reverse Proxy (Nginx)
 - Actual application server

```
Client → CDN → API Gateway → Load Balancer → App Server 1
                                           → App Server 2
```

Constraint 6: Code on Demand (Optional)

- **Definition:** Servers can optionally extend client functionality by transferring executable code (e.g., JavaScript).
- This is the only optional constraint.
- Example: A server sends JavaScript widgets or applets that the client executes.

- In modern REST APIs, this is rarely used explicitly but relates to how SPAs work.

Summary Table: REST Constraints

#	Constraint	Key Benefit
1	Client-Server	Separation of concerns, independent evolution
2	Stateless	Scalability, reliability
3	Cacheable	Performance, reduced server load
4	Uniform Interface	Simplicity, discoverability
5	Layered System	Security, scalability, flexibility
6	Code on Demand	Extensibility (optional)

3. RESTful API Design

HTTP Methods (CRUD Mapping)

HTTP Method	CRUD Operation	Description	Idempotent?	Safe?
GET	Read	Retrieve resource(s)	Yes	Yes
POST	Create	Create a new resource	No	No
PUT	Update (Full)	Replace entire resource	Yes	No
PATCH	Update (Partial)	Modify part of resource	No	No
DELETE	Delete	Remove resource	Yes	No

Idempotent: Same request made multiple times produces the same result. **Safe:** Does not modify server state.

Resource Naming Conventions

Good RESTful URL Design:

```
# Collections (plural nouns)
GET    /api/users      → List all users
POST   /api/users      → Create a new user

# Individual resources (noun + ID)
GET    /api/users/42   → Get user with ID 42
PUT    /api/users/42   → Replace user 42 completely
PATCH /api/users/42   → Update user 42 partially
DELETE /api/users/42   → Delete user 42

# Nested resources (relationships)
GET    /api/users/42/orders → All orders of user 42
GET    /api/users/42/orders/7 → Order 7 of user 42
POST   /api/users/42/orders → Create order for user 42
```

Bad URL Design (Anti-patterns to avoid):

GET /getUsers	← Verb in URL (wrong!)
POST /createUser	← Verb in URL (wrong!)
GET /user/42/getOrders	← Verb in URL (wrong!)
GET /api/User/42	← PascalCase (avoid, use lowercase)

HTTP Status Codes

Range	Category	Common Codes
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Redirection	301 Moved Permanently, 304 Not Modified
4xx	Client Error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 405 Method Not Allowed, 422 Unprocessable Entity, 429 Too Many Requests
5xx	Server Error	500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable

Key Status Codes to Know:

- 200 OK — Successful GET, PUT, PATCH
- 201 Created — Successful POST (new resource created)
- 204 No Content — Successful DELETE (no body to return)
- 400 Bad Request — Invalid request syntax or parameters
- 401 Unauthorized — Not authenticated (no or invalid token)
- 403 Forbidden — Authenticated but not authorized
- 404 Not Found — Resource does not exist
- 409 Conflict — Resource already exists (duplicate)
- 422 Unprocessable Entity — Validation error
- 500 Internal Server Error — Server-side bug

API Versioning

Why version APIs? To allow the API to evolve without breaking existing clients.

Versioning Strategies:

1. URL Path Versioning (Most Common)

/api/v1/users
/api/v2/users

2. Query Parameter Versioning

/api/users?version=1
/api/users?v=2

3. Header Versioning

```
GET /api/users
Accept: application/vnd.myapp.v2+json
```

4. Subdomain Versioning

```
v1.api.example.com/users
v2.api.example.com/users
```

Best Practice: URL path versioning is most explicit and widely used. Always version from day one.

E-Commerce API Example

```
# Product Management
GET    /api/v1/products           → List products (with filters, pagination)
GET    /api/v1/products/123    → Get product details
POST   /api/v1/products        → Create product (admin only)
PUT    /api/v1/products/123    → Update product (admin only)
DELETE /api/v1/products/123     → Delete product (admin only)
GET    /api/v1/products?category=electronics&min_price=100

# Cart & Orders
GET    /api/v1/cart            → View cart
POST   /api/v1/cart/items      → Add item to cart
DELETE /api/v1/cart/items/456  → Remove item
POST   /api/v1/orders          → Place order (checkout)
GET    /api/v1/orders          → My orders
GET    /api/v1/orders/789      → Order details
PATCH /api/v1/orders/789/cancel → Cancel order
```

Query Parameters for Filtering, Sorting, Pagination

```
# Filtering
GET /api/v1/products?category=electronics&brand=samsung

# Sorting
GET /api/v1/products?sort=price&order=asc
GET /api/v1/products?sort=-created_at (minus = descending)

# Pagination
GET /api/v1/products?page=2&limit=20
GET /api/v1/products?offset=40&limit=20

# Search
GET /api/v1/products?search=laptop
```

Pagination Response Format:

```
{
  "count": 150,
  "next": "/api/v1/products?page=3&limit=20",
```

```
"previous": "/api/v1/products?page=1&limit=20",
"results": [...]
}
```

REST API Best Practices

1. **Use nouns, not verbs** in URLs
2. **Use plural nouns** for collections (`/users` , not `/user`)
3. **Use HTTPS** — always encrypt API traffic
4. **Use proper HTTP methods** — don't use GET for mutations
5. **Return appropriate status codes** — be precise
6. **Version your API** from day one
7. **Use JSON** as the primary data format
8. **Implement pagination** for list endpoints
9. **Document your API** (Swagger/OpenAPI)
10. **Implement rate limiting** — prevent abuse
11. **Use authentication** — JWT or OAuth2
12. **Handle errors gracefully** — return consistent error format
13. **Use filtering, sorting, searching** via query parameters

Consistent Error Response Format:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid email format",
    "field": "email",
    "status": 422
  }
}
```

4. JSON and XML

JSON (JavaScript Object Notation)

JSON is a lightweight, text-based data interchange format. Easy for humans to read and write; easy for machines to parse and generate.

JSON Data Types:

- String: `"hello"`
- Number: `42` , `3.14`
- Boolean: `true` , `false`
- Null: `null`
- Array: `[1, 2, 3]`
- Object: `{"key": "value"}`

JSON Example:

```
{
  "student": {
```

```

    "id": 1042,
    "name": "Ramesh Sharma",
    "age": 21,
    "enrolled": true,
    "courses": ["Web Programming", "Database Systems", "Algorithms"],
    "address": {
      "city": "Kathmandu",
      "district": "Bagmati",
      "country": "Nepal"
    },
    "gpa": 3.75,
    "advisor": null
  }
}

```

JSON Rules:

- Keys must be strings in double quotes
- No comments allowed
- No trailing commas
- Strings must use double quotes (not single quotes)

XML (eXtensible Markup Language)

XML uses tags to define structure, similar to HTML but for data representation.

Same data in XML:

```

<?xml version="1.0" encoding="UTF-8"?>
<student>
  <id>1042</id>
  <name>Ramesh Sharma</name>
  <age>21</age>
  <enrolled>true</enrolled>
  <courses>
    <course>Web Programming</course>
    <course>Database Systems</course>
    <course>Algorithms</course>
  </courses>
  <address>
    <city>Kathmandu</city>
    <district>Bagmati</district>
    <country>Nepal</country>
  </address>
  <gpa>3.75</gpa>
  <advisor></advisor>
</student>

```

JSON vs XML Comparison

Feature	JSON	XML
---------	------	-----

Verbosity	Compact, less verbose	More verbose (opening + closing tags)
Readability	Human-friendly	Readable but bulkier
Data Types	Native support (number, boolean, null, array)	Everything is text; need attributes or schemas
Parsing	Fast, native in JavaScript (JSON.parse)	Slower, needs XML parser (DOM/SAX)
Comments	Not supported	Supported (<!-- comment -->)
Attributes	Not applicable	Supports tag attributes
Schema Validation	JSON Schema	DTD, XSD (more mature)
Namespace Support	Not supported	Supported
File Size	Smaller	Larger (redundant tags)
Browser Support	Native in all browsers	Needs XMLHttpRequest/DOMParser
Use Cases	REST APIs, web apps, config files	SOAP APIs, enterprise, document formats
Array Support	Natural [] syntax	Requires repeated tags

File Size Comparison (same data):

- JSON: ~180 bytes
- XML: 350 bytes (2x larger)

When to Use Which?

- **Use JSON:** REST APIs, web/mobile apps, JavaScript frontends, configuration (package.json)
- **Use XML:** SOAP/enterprise web services, document formats (Word, SVG, RSS), complex validation needs, legacy system integration

Parsing in Python

```
import json
import xml.etree.ElementTree as ET

# JSON Parsing
json_string = '{"name": "Alice", "age": 25}'
data = json.loads(json_string)          # string → dict
print(data["name"])                     # Alice

json_output = json.dumps(data, indent=2) # dict → string

# XML Parsing
xml_string = "<student><name>Alice</name><age>25</age></student>"
root = ET.fromstring(xml_string)
print(root.find("name").text)           # Alice
```

```
# XML to dict (manual)
student = {child.tag: child.text for child in root}
```

5. Data Validation and Serialization

Serialization

Serialization is the process of converting a complex data structure (like a Python object/queryset) into a format (JSON/XML) that can be transmitted over a network or stored.

Deserialization is the reverse — converting JSON/XML data received from the client back into a Python object that can be saved to the database.

```
[Python Object / DB Model] → Serialize → [JSON Response]
      ↑
[JSON Request] → Deserialize → [Python Object]
```

Django REST Framework (DRF) Serializers

DRF serializers handle both serialization AND validation.

Basic Serializer:

```
# serializers.py
from rest_framework import serializers
from .models import Product

class ProductSerializer(serializers.ModelSerializer):
    class Meta:
        model = Product
        fields = ['id', 'name', 'price', 'category', 'stock', 'created_at']
        read_only_fields = ['id', 'created_at']

    # Custom field-level validation
    def validate_price(self, value):
        if value <= 0:
            raise serializers.ValidationError("Price must be positive.")
        return value

    def validate_stock(self, value):
        if value < 0:
            raise serializers.ValidationError("Stock cannot be negative.")
        return value

    # Object-level validation (cross-field)
    def validate(self, data):
        if data.get('category') == 'luxury' and data.get('price') < 1000:
            raise serializers.ValidationError(
                "Luxury products must cost at least 1000."
            )
        return data
```

Using the Serializer in a View:

```
# views.py
from rest_framework.decorators import api_view
from rest_framework.response import Response
from rest_framework import status
from .models import Product
from .serializers import ProductSerializer

@api_view(['GET', 'POST'])
def product_list(request):
    if request.method == 'GET':
        products = Product.objects.all()
        serializer = ProductSerializer(products, many=True) # Serialize queryset
        return Response(serializer.data, status=status.HTTP_200_OK)

    elif request.method == 'POST':
        serializer = ProductSerializer(data=request.data) # Deserialize
        if serializer.is_valid(): # Validate
            serializer.save() # Save to DB
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

@api_view(['GET', 'PUT', 'DELETE'])
def product_detail(request, pk):
    try:
        product = Product.objects.get(pk=pk)
    except Product.DoesNotExist:
        return Response(status=status.HTTP_404_NOT_FOUND)

    if request.method == 'GET':
        serializer = ProductSerializer(product)
        return Response(serializer.data)

    elif request.method == 'PUT':
        serializer = ProductSerializer(product, data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    elif request.method == 'DELETE':
        product.delete()
        return Response(status=status.HTTP_204_NO_CONTENT)
```

DRF Class-Based Views (ViewSet)

```
# views.py
from rest_framework import viewsets
```

```

from rest_framework.permissions import IsAuthenticated

class ProductViewSet(viewsets.ModelViewSet):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
    permission_classes = [IsAuthenticated]

    def get_queryset(self):
        queryset = super().get_queryset()
        category = self.request.query_params.get('category')
        if category:
            queryset = queryset.filter(category=category)
        return queryset

# urls.py
from rest_framework.routers import DefaultRouter

router = DefaultRouter()
router.register('products', ProductViewSet)
urlpatterns = router.urls
# Automatically creates:
# GET/POST /products/
# GET/PUT/PATCH/DELETE /products/{id}/

```

Data Validation

Validation ensures data conforms to expected formats, types, ranges, and business rules before processing or storing.

Levels of Validation:

1. **Client-side validation** (HTML5, JavaScript) — immediate feedback, but NOT trustworthy
2. **Server-side validation** (serializers, forms) — always required, authoritative
3. **Database-level validation** (constraints, NOT NULL, UNIQUE) — last line of defense

DRF Built-in Validators:

```

from rest_framework import serializers
from django.core.validators import MinValueValidator, MaxValueValidator

class ReviewSerializer(serializers.ModelSerializer):
    rating = serializers.IntegerField(
        validators=[MinValueValidator(1), MaxValueValidator(5)]
    )
    email = serializers.EmailField()
    url = serializers.URLField(required=False)

    class Meta:
        model = Review
        fields = ['rating', 'email', 'url', 'comment']

```

Custom Validators:

```

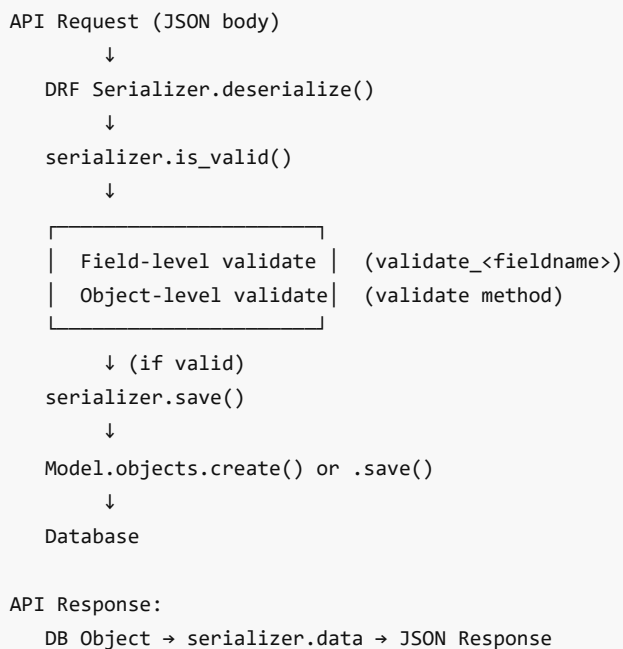
from rest_framework import serializers
import re

def nepali_phone_validator(value):
    """Validate Nepali phone numbers: 98XXXXXXX or 97XXXXXXX"""
    pattern = r'^(98|97)\d{8}$'
    if not re.match(pattern, str(value)):
        raise serializers.ValidationError(
            "Enter a valid Nepali phone number (e.g., 9841234567)"
        )

class UserSerializer(serializers.ModelSerializer):
    phone = serializers.CharField(validators=[nepali_phone_validator])

```

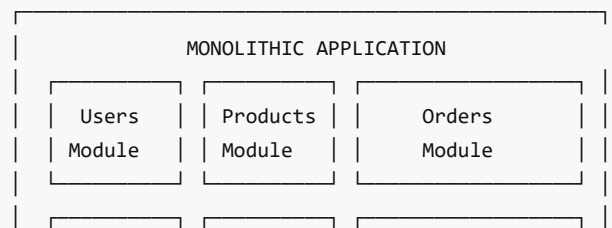
Serialization Flow Diagram



6. Microservices Architecture

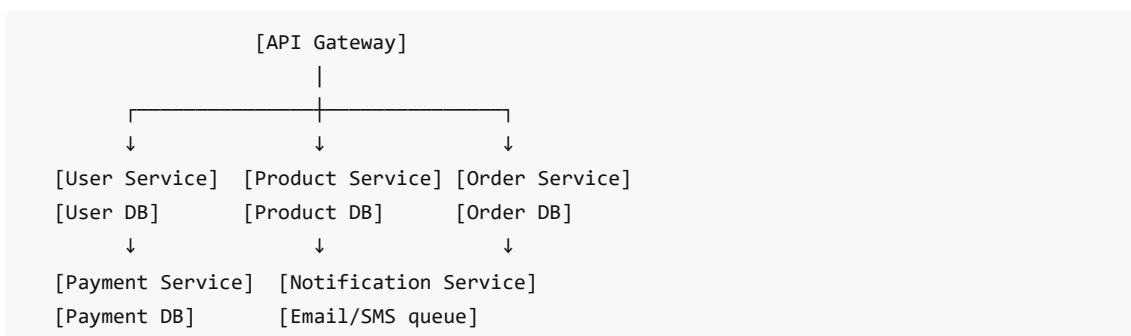
Monolithic vs Microservices

Monolithic Architecture: All application components (user management, products, orders, payments, notifications) are part of ONE large codebase and deployed as a single unit.





Microservices Architecture: The application is broken into small, independent services, each responsible for a specific business function, with its own database.



Characteristics of Microservices

1. **Single Responsibility:** Each service handles one business domain
2. **Independent Deployment:** Deploy one service without affecting others
3. **Own Data Store:** Each service has its own database (polyglot persistence)
4. **Communicate via APIs:** Services talk via REST, gRPC, or message queues
5. **Decentralized Governance:** Different teams can use different technologies
6. **Fault Isolation:** Failure in one service doesn't crash the whole system
7. **Scalability:** Scale individual services independently based on demand

Advantages of Microservices

Advantage	Explanation
Independent Scalability	Scale only the Product service during a sale, not the whole app
Technology Flexibility	User service in Python, Recommendation service in Go
Faster Deployment	Deploy the Payment service fix without touching Orders
Fault Isolation	If Notification service crashes, orders still work
Team Autonomy	Different teams own different services independently
Easy to Understand	Each service is small and focused

Challenges of Microservices

Challenge	Explanation
Distributed System Complexity	Network calls can fail; need retries, circuit breakers
Data Consistency	No single database transaction across services (eventual consistency)
Service Discovery	How does Service A find Service B? (need service registry like Consul)

Inter-Service Communication	REST/gRPC overhead; message queue management
Testing Complexity	Integration testing across many services is hard
Operational Overhead	Need Docker, Kubernetes, monitoring for dozens of services
Debugging	Tracing a request across 5 services is difficult (need distributed tracing)

Microservices Communication Patterns

Synchronous (Request-Response):

```
# Service A calls Service B directly
import requests

def get_user_orders(user_id):
    # Order service calls User service
    response = requests.get(f"http://user-service/api/users/{user_id}")
    if response.status_code == 200:
        return response.json()
    raise Exception("User service unavailable")
```

Asynchronous (Message Queue):

```
# When order is placed, publish event to queue
# Notification service subscribes and sends email

# Producer (Order Service)
import pika

connection = pika.BlockingConnection(pika.ConnectionParameters('rabbitmq'))
channel = connection.channel()
channel.queue_declare(queue='order_placed')
channel.basic_publish(
    exchange='',
    routing_key='order_placed',
    body='{"order_id": 123, "user_email": "user@example.com"}'
)

# Consumer (Notification Service)
def callback(ch, method, properties, body):
    data = json.loads(body)
    send_email(data['user_email'], f"Order {data['order_id']} confirmed!")

channel.basic_consume(queue='order_placed', on_message_callback=callback)
channel.start_consuming()
```

API Gateway Pattern

An API Gateway is a single entry point for all client requests. It routes requests to the appropriate microservice.

Responsibilities of API Gateway:

- Request routing
- Authentication/Authorization
- Rate limiting
- SSL termination
- Load balancing
- Request/response transformation
- Logging and monitoring

```
Client (Browser/Mobile)
  ↓
[API Gateway]      ← Auth check, rate limiting, routing
  ↓ routes to:
/api/users/* → User Service
/api/products/* → Product Service
/api/orders/* → Order Service
```

7. Building and Testing REST APIs

Complete Django REST Framework API Example

Project Setup:

```
pip install django djangorestframework
django-admin startproject myapi
cd myapi
python manage.py startapp store
```

Models:

```
# store/models.py
from django.db import models

class Category(models.Model):
    name = models.CharField(max_length=100)

    def __str__(self):
        return self.name

class Product(models.Model):
    name = models.CharField(max_length=200)
    description = models.TextField(blank=True)
    price = models.DecimalField(max_digits=10, decimal_places=2)
    category = models.ForeignKey(Category, on_delete=models.CASCADE,
related_name='products')
    stock = models.IntegerField(default=0)
    created_at = models.DateTimeField(auto_now_add=True)
    is_active = models.BooleanField(default=True)
```



```
def __str__(self):
    return self.name
```

Serializers:

```
# store/serializers.py
from rest_framework import serializers
from .models import Product, Category

class CategorySerializer(serializers.ModelSerializer):
    product_count = serializers.SerializerMethodField()

    class Meta:
        model = Category
        fields = ['id', 'name', 'product_count']

    def get_product_count(self, obj):
        return obj.products.count()

class ProductSerializer(serializers.ModelSerializer):
    category_name = serializers.CharField(source='category.name', read_only=True)

    class Meta:
        model = Product
        fields = ['id', 'name', 'description', 'price', 'category', 'category_name',
                  'stock', 'created_at', 'is_active']
        read_only_fields = ['id', 'created_at']

    def validate_price(self, value):
        if value <= 0:
            raise serializers.ValidationError("Price must be greater than zero.")
        return value
```

Views:

```
# store/views.py
from rest_framework import generics, status, filters
from rest_framework.response import Response
from rest_framework.permissions import IsAuthenticatedOrReadOnly
from django_filters.rest_framework import DjangoFilterBackend
from .models import Product, Category
from .serializers import ProductSerializer, CategorySerializer

class ProductListCreateView(generics.ListCreateAPIView):
    queryset = Product.objects.filter(is_active=True)
    serializer_class = ProductSerializer
    permission_classes = [IsAuthenticatedOrReadOnly]
    filter_backends = [DjangoFilterBackend, filters.SearchFilter, filters.OrderingFilter]
    filterset_fields = ['category']
    search_fields = ['name', 'description']
    ordering_fields = ['price', 'created_at']
```

```
class ProductDetailView(generics.RetrieveUpdateDestroyAPIView):
    queryset = Product.objects.all()
    serializer_class = ProductSerializer
    permission_classes = [IsAuthenticatedOrReadOnly]
```

URLs:

```
# store/urls.py
from django.urls import path
from . import views

urlpatterns = [
    path('products/', views.ProductListCreateView.as_view(), name='product-list'),
    path('products/<int:pk>', views.ProductDetailView.as_view(), name='product-detail'),
]

# myapi/urls.py
from django.urls import path, include

urlpatterns = [
    path('api/v1/', include('store.urls')),
    path('api/v1/auth/', include('rest_framework.urls')),
]
```

API Authentication with JWT

```
# settings.py
INSTALLED_APPS = [
    ...
    'rest_framework',
    'rest_framework_simplejwt',
]

REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': [
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ],
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ],
}

# urls.py
from rest_framework_simplejwt.views import TokenObtainPairView, TokenRefreshView

urlpatterns += [
    path('api/token/', TokenObtainPairView.as_view(), name='token_obtain_pair'),
```

```
    path('api/token/refresh/', TokenRefreshView.as_view(), name='token_refresh'),
]
```

JWT Auth Flow:

```
# 1. Login to get tokens
POST /api/token/
Content-Type: application/json
{"username": "admin", "password": "password123"}

→ Response:
{"access": "eyJ0eXA...", "refresh": "eyJ0eXA..."}

# 2. Use access token in subsequent requests
GET /api/v1/products/
Authorization: Bearer eyJ0eXA...

# 3. Refresh expired access token
POST /api/token/refresh/
{"refresh": "eyJ0eXA..."}
→ {"access": "new_access_token..."}
```

Testing APIs

Manual Testing with curl:

```
# GET all products
curl http://localhost:8000/api/v1/products/

# POST create product (with auth)
curl -X POST http://localhost:8000/api/v1/products/ \
  -H "Authorization: Bearer <token>" \
  -H "Content-Type: application/json" \
  -d '{"name": "Laptop", "price": 75000, "category": 1, "stock": 10}'

# PUT update
curl -X PUT http://localhost:8000/api/v1/products/1/ \
  -H "Authorization: Bearer <token>" \
  -H "Content-Type: application/json" \
  -d '{"name": "Laptop Pro", "price": 85000, "category": 1, "stock": 5}'

# DELETE
curl -X DELETE http://localhost:8000/api/v1/products/1/ \
  -H "Authorization: Bearer <token>"
```

Automated Testing with Django Test Client:

```
# store/tests.py
from django.test import TestCase
from rest_framework.test import APITestCase, APIClient
```

```

from rest_framework import status
from django.contrib.auth.models import User
from .models import Product, Category

class ProductAPITest(APITestCase):
    def setUp(self):
        self.user = User.objects.create_user(username='testuser', password='pass123')
        self.category = Category.objects.create(name='Electronics')
        self.product = Product.objects.create(
            name='Test Phone', price=25000, category=self.category, stock=10
        )
        self.client.force_authenticate(user=self.user)

    def test_get_product_list(self):
        response = self.client.get('/api/v1/products/')
        self.assertEqual(response.status_code, status.HTTP_200_OK)
        self.assertEqual(len(response.data['results']), 1)

    def test_create_product(self):
        data = {'name': 'New Phone', 'price': 30000, 'category': self.category.id, 'stock':
5}

        response = self.client.post('/api/v1/products/', data)
        self.assertEqual(response.status_code, status.HTTP_201_CREATED)
        self.assertEqual(Product.objects.count(), 2)

    def test_create_product_invalid_price(self):
        data = {'name': 'Bad Product', 'price': -100, 'category': self.category.id}
        response = self.client.post('/api/v1/products/', data)
        self.assertEqual(response.status_code, status.HTTP_400_BAD_REQUEST)

    def test_unauthenticated_create(self):
        self.client.force_authenticate(user=None)
        data = {'name': 'Laptop', 'price': 50000, 'category': self.category.id}
        response = self.client.post('/api/v1/products/', data)
        self.assertEqual(response.status_code, status.HTTP_401_UNAUTHORIZED)

```

8. Model Exam Questions & Answers

Short Answer Questions

Q1. What is an API? List its roles in web development. [2071, 2072, 2074]

Answer:

An **API (Application Programming Interface)** is a set of rules and protocols that enables different software applications to communicate with each other. It defines how requests and responses are structured.

Roles of APIs in web development:

1. **Frontend-Backend Communication:** Connect React/Angular frontend to Django/Node backend
2. **Third-party Integration:** Payment gateways (eSewa, Stripe), maps, social login
3. **Mobile app backend:** Mobile apps consume same REST APIs as web

- 4. **Microservices communication:** Internal service-to-service communication
- 5. **Public data access:** Weather, currency, government data
- 6. **Business logic exposure:** Share capabilities with partners via APIs

Q2. List and explain the six constraints of REST architecture. [2073, 2075, 2076, 2078]

Answer:

The six REST constraints defined by Roy Fielding are:

1. **Client-Server:** Separation of UI (client) and data storage (server). Enables independent evolution of both.
2. **Stateless:** Each request must be self-contained. The server stores no client session state between requests. Authentication info (JWT token) must be sent with every request.
3. **Cacheable:** Responses must declare whether they are cacheable. Caching reduces server load and improves performance.
4. **Uniform Interface:** A consistent way to interact with the server — use standard HTTP methods, resource URIs, and self-descriptive messages. Includes HATEOAS (links to related resources in responses).
5. **Layered System:** Client cannot tell whether it is connected directly to the server or through intermediaries (load balancer, cache, API gateway). Improves scalability and security.
6. **Code on Demand (Optional):** Servers may extend client functionality by sending executable code (JavaScript). The only optional constraint.

Q3. Differentiate between JSON and XML with examples. [2072, 2074, 2077]

Answer:

Feature	JSON	XML
Verbosity	Compact	Verbose
Data types	Native (number, boolean, null, array)	Everything is text
Comments	Not supported	Supported
Parsing speed	Fast	Slower
File size	Smaller	Larger (~2x)
Use case	REST APIs, modern web	SOAP, enterprise, legacy

JSON Example:

```
{ "user": { "name": "Ram", "age": 22, "active": true } }
```

XML Example:

```
<user><name>Ram</name><age>22</age><active>true</active></user>
```

JSON is preferred for modern REST APIs due to its simplicity, native JavaScript support, and smaller payload size.

Q4. What is serialization and deserialization? Explain with a Django REST Framework example. [2075, 2079]

Answer:

Serialization is converting a Python object (Django model instance or queryset) into a transmittable format (JSON/XML) for API responses.

Deserialization is the reverse — converting incoming JSON data (from HTTP request body) into a Python object that can be validated and saved to the database.

DRF Example:

```
from rest_framework import serializers
from .models import Product

class ProductSerializer(serializers.ModelSerializer):
    class Meta:
        model = Product
        fields = ['id', 'name', 'price']

# Serialization (Python → JSON):
product = Product.objects.get(id=1)
serializer = ProductSerializer(product)
print(serializer.data) # {'id': 1, 'name': 'Laptop', 'price': '75000.00'}

# Deserialization (JSON → Python):
data = {'name': 'New Laptop', 'price': '80000.00'}
serializer = ProductSerializer(data=data)
if serializer.is_valid():
    serializer.save() # Creates new Product in database
```

Q5. What are microservices? Compare with monolithic architecture. [2076, 2078, 2080]

Answer:

Microservices is an architectural style where an application is built as a collection of small, independent services, each responsible for a specific business function, communicating via APIs.

Comparison:

Aspect	Monolithic	Microservices
Deployment	Single unit	Independent services
Scaling	Scale entire app	Scale individual services
Technology	Single tech stack	Polyglot (each service can differ)
Failure	One crash affects all	Fault isolation
Complexity	Simpler initially	Complex distributed system
Team size	Small teams work well	Scales with large teams

Database	Shared database	Each service has own DB
----------	-----------------	-------------------------

When to use Microservices: Large, complex applications with multiple teams and high scalability needs (Netflix, Amazon). **When to use Monolithic:** Startups, small teams, simple applications in early stages.

Long Answer Questions

Q6. Design a RESTful API for a library management system. Show endpoints, HTTP methods, status codes, and sample request/response. [2073, 2077, 2079]

Answer:

Library Management REST API Design:

Base URL: `https://library.ncoe.edu.np/api/v1/`

Resources and Endpoints:

```
Books:
GET    /books           → List all books
POST   /books           → Add new book (librarian only)
GET    /books/{id}      → Get book details
PUT    /books/{id}      → Update book (librarian only)
DELETE /books/{id}      → Delete book (admin only)
GET    /books?search=python&available=true → Search with filters

Members:
GET    /members         → List members
POST   /members         → Register new member
GET    /members/{id}    → Get member info
PUT    /members/{id}    → Update member

Borrowings:
POST   /borrowings   → Borrow a book
GET    /borrowings?member_id=5 → Member's borrowing history
PATCH /borrowings/{id}/return → Return a book
GET    /borrowings/overdue → List overdue books
```

Sample: Borrow a Book

```
POST /api/v1/borrowings/
Authorization: Bearer <member_token>
Content-Type: application/json

{
  "book_id": 42,
  "member_id": 7
}
```

Success Response (201 Created):

```
{
  "id": 156,
  "book": {"id": 42, "title": "Clean Code", "isbn": "978-0132350884"},
  "member": {"id": 7, "name": "Sita Rai"},
  "borrowed_date": "2026-03-18",
  "due_date": "2026-04-01",
  "status": "borrowed"
}
```

Error Responses:

```
// 404 - Book not found
{"error": {"code": "NOT_FOUND", "message": "Book with id 42 not found"}}

// 409 - Book already borrowed
{"error": {"code": "CONFLICT", "message": "This book is currently not available"}}

// 422 - Validation error
{"error": {"code": "VALIDATION_ERROR", "message": "member_id is required"}}
```

HTTP Status Codes used:

- 200 OK — successful GET
- 201 Created — successful borrow/create
- 204 No Content — successful delete
- 400 Bad Request — malformed request
- 401 Unauthorized — not logged in
- 403 Forbidden — insufficient permissions
- 404 Not Found — book/member doesn't exist
- 409 Conflict — book already borrowed
- 422 Unprocessable Entity — validation failed

Q7. Explain microservices architecture in detail including advantages, challenges, and communication patterns. [2078, 2080]

Answer:

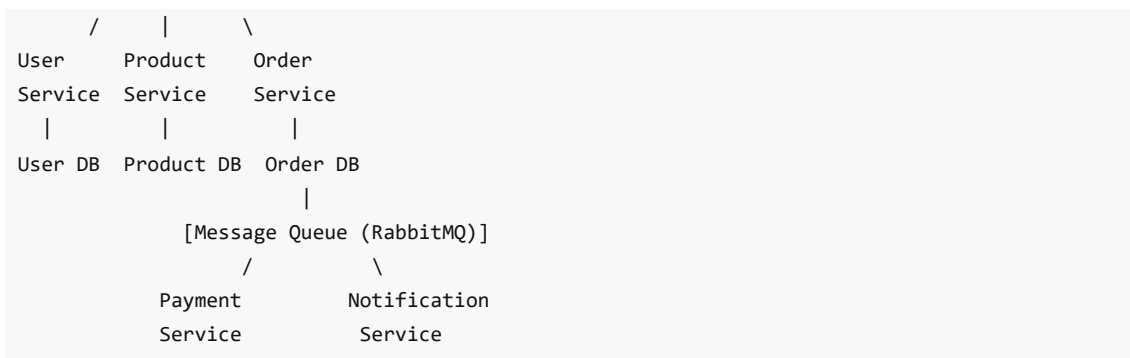
Microservices Architecture decomposes a monolithic application into small, loosely coupled services.

Core Characteristics:

- Each service handles one business capability
- Independent deployment and scaling
- Own database per service (polyglot persistence)
- Communicate via REST APIs or message queues
- Built and maintained by small, autonomous teams

Architecture Diagram:

```
Client (Browser/Mobile App)
    ↓
[API Gateway]
```

Advantages:

1. Each service can be scaled independently
2. Fault in one service doesn't crash the system
3. Different teams can use different technologies
4. Faster, independent deployments
5. Easier to understand individual services
6. Better alignment with business domains

Challenges:

1. Distributed system complexity (network failures, latency)
2. Data consistency across services (no distributed transactions)
3. Service discovery — how services find each other
4. Increased operational overhead (Docker, Kubernetes, monitoring)
5. Debugging distributed requests (need distributed tracing with Jaeger/Zipkin)
6. Testing complexity (need service stubs/mocks)

Communication Patterns:

- **Synchronous:** REST API calls between services (immediate response needed)
- **Asynchronous:** Message queues (RabbitMQ, Kafka) — order placed → notification service sends email
- **Event-driven:** Services publish/subscribe to events

When Microservices Make Sense:

- Large teams (10+ developers)
- Complex domains with clear service boundaries
- High scalability requirements
- Independent release cycles needed (Netflix, Amazon use microservices)

Q8. What is data validation? How does Django REST Framework implement validation? Provide examples of field-level and object-level validation. [2074, 2079]

Answer:

Data Validation is the process of verifying that incoming data meets expected constraints before it is processed or stored. It ensures data integrity, security, and business rule compliance.

Why Validation is Important:

- Prevents invalid data from entering the database
- Protects against injection attacks
- Provides meaningful error messages to clients

- Enforces business rules

DRF Validation Levels:

1. Field-Level Validation (per-field, using `validate_<fieldname>` method):

```
class UserSerializer(serializers.ModelSerializer):
    class Meta:
        model = User
        fields = ['username', 'email', 'age', 'phone']

    def validate_age(self, value):
        if value < 18:
            raise serializers.ValidationError("Must be at least 18 years old.")
        if value > 120:
            raise serializers.ValidationError("Invalid age.")
        return value

    def validate_phone(self, value):
        if not value.startswith(('98', '97')):
            raise serializers.ValidationError("Must be a valid Nepali phone number.")
        if len(value) != 10:
            raise serializers.ValidationError("Phone must be 10 digits.")
        return value
```

2. Object-Level Validation (cross-field validation, using `validate` method):

```
class EventSerializer(serializers.ModelSerializer):
    class Meta:
        model = Event
        fields = ['title', 'start_date', 'end_date', 'ticket_price', 'event_type']

    def validate(self, data):
        # Cross-field: end must be after start
        if data['end_date'] <= data['start_date']:
            raise serializers.ValidationError(
                "End date must be after start date."
            )
        # Business rule: premium events must cost at least 500
        if data['event_type'] == 'premium' and data['ticket_price'] < 500:
            raise serializers.ValidationError(
                "Premium events must have ticket price of at least Rs. 500."
            )
        return data
```

3. Built-in DRF Validators:

```
from rest_framework.validators import UniqueValidator
from django.core.validators import MinValueValidator

class ProductSerializer(serializers.ModelSerializer):
```

```

name = serializers.CharField(
    validators=[UniqueValidator(queryset=Product.objects.all())]
)
price = serializers.DecimalField(
    max_digits=10,
    decimal_places=2,
    validators=[MinValueValidator(0.01)]
)

```

Validation Error Response:

```

{
  "age": ["Must be at least 18 years old."],
  "phone": ["Phone must be 10 digits."],
  "non_field_errors": ["End date must be after start date."]
}

```

Q9. Compare REST and SOAP web services. What makes REST the preferred choice for modern web APIs? [2071, 2073]

Answer:

Feature	REST	SOAP
Protocol	HTTP	HTTP, SMTP, TCP (any)
Data format	JSON, XML	XML only
Complexity	Simple	Complex (WSDL, envelope)
Performance	Faster, lightweight	Slower (XML overhead)
State	Stateless	Can be stateful
Security	HTTPS, JWT, OAuth	WS-Security (built-in)
Caching	Supports HTTP caching	Difficult to cache
Error handling	HTTP status codes	SOAP Fault element
Standards	Architectural style	Strict standard (W3C)
Use case	Modern web/mobile APIs	Enterprise, banking, legacy

Why REST is preferred for modern APIs:

1. **Simplicity:** Uses familiar HTTP methods; no complex XML envelopes or WSDL contracts required
2. **JSON support:** Lighter payload than SOAP's mandatory XML; faster parsing in browsers
3. **HTTP caching:** GET responses can be cached naturally; SOAP has no native caching
4. **Statelessness:** Each request is independent; better for horizontal scaling
5. **Mobile-friendly:** Lower bandwidth consumption; critical for mobile apps
6. **Developer experience:** Easy to test with curl/Postman; widely understood

SOAP remains relevant in enterprise/financial systems where strict contracts, ACID compliance, and built-in WS-Security are required.

Q10. What is API versioning? Why is it needed, and what are the different versioning strategies? [2076, 2080]

Answer:

API Versioning is the practice of managing changes to an API over time while maintaining backward compatibility for existing clients.

Why Versioning is Needed:

- APIs evolve — new features added, old ones deprecated
- Breaking changes (removing fields, changing data types) would break existing clients
- Different clients may need different versions (mobile app v1 vs web app v2)
- Allows gradual migration to new API versions

Versioning Strategies:

1. URL Path Versioning (Most Common):

```
GET /api/v1/products/  
GET /api/v2/products/ ← v2 may return additional fields
```

Pros: Explicit, easy to test, visible in browser *Cons:* URL proliferation with many versions

2. Query Parameter Versioning:

```
GET /api/products?version=1  
GET /api/products?v=2
```

Pros: Clean URLs, easy to implement *Cons:* Can be overlooked, not RESTful

3. Request Header Versioning:

```
GET /api/products/  
Accept: application/vnd.myapp.v2+json
```

Pros: Clean URLs, follows HTTP spec *Cons:* Harder to test in browser

4. Subdomain Versioning:

```
https://v1.api.example.com/products/  
https://v2.api.example.com/products/
```

Pros: Clear separation *Cons:* DNS configuration required, more complex

Best Practices:

- Version from day one (even v1)
 - Never break existing versions without notice
 - Provide deprecation timeline (e.g., v1 deprecated in 6 months)
 - Document changes between versions
 - URL path versioning is generally recommended for clarity
-

End of Chapter 4 Notes

Total Chapter Coverage: API fundamentals, 6 REST constraints, RESTful design (HTTP methods, status codes, URL naming, versioning), JSON vs XML comparison, DRF serialization/validation, microservices architecture, building and testing REST APIs with complete Django examples.